Hadoop - Travaux pratiques TP 0 à TP 3

Compte rendu, solutions et dossier reproductible - 20 octobre 2026

État de réalisation et preuves

Les 28 exercices numérotés sont traités ci-dessous. Les programmes Java, la configuration Docker, une variante WSL, les scripts de lancement, les données et les résultats locaux sont joints.

Limite d'exécution : le cluster Hadoop n'a pas pu être démarré dans cet environnement. Le client Docker est installé, mais son moteur Linux ne répond pas. Les tentatives de récupération de Hadoop ont été interrompues par des erreurs réseau. Les résultats numériques WordCount et amis communs ont été recalculés réellement avec le programme Python indépendant fourni. Ils ne constituent pas une exécution MapReduce distribuée. Les commandes HDFS/YARN ci-dessous sont des procédures à exécuter ; les valeurs annoncées pour ces services sont des réponses théoriques ou les références du sujet, pas des observations inventées.

Les captures du dossier `captures` montrent les pages de vérification locale affichées dans un navigateur. Elles sont explicitement légendées. Aucune capture de NameNode, ResourceManager, application YARN ou terminal Hadoop réussi n'est fabriquée. Le JAR compilé et les captures du cluster restent à produire sur une installation fonctionnelle.

Le sujet est joint comme référence pédagogique. Les procédures qui y figurent définissent les exercices ; elles ne constituent pas une autorisation de modifier d'autres fichiers ou services de l'utilisateur.

Choix du jeu de données

Le dossier `donnees/livres` contient **six** fichiers, malgré certaines mentions de « cinq livres » dans le fichier LISEZMOI. Le corpus principal comprend uniquement Alice, Hamlet, Moby Dick, Emma et Father Brown. La Bible est réservée à l'exercice de découpage en blocs. Le script charge explicitement les cinq fichiers, sans joker `*.txt`.

Le corpus principal représente **2 843 966 octets et 55 654 lignes**. Le dossier `gutenberg` contient bien 18 livres. L'ajout de `THE END` est effectué après les WordCount afin de conserver le corpus de référence.

TP 0 - Installation et configuration

Architecture et mise en route

Le NameNode conserve l'espace de noms et les métadonnées des blocs ; les DataNodes conservent les données. Le ResourceManager distribue les ressources ; les NodeManagers lancent les containers. Un ApplicationMaster est créé pour chaque application YARN.

L'option Docker du sujet est fournie dans `docker-compose.yml` et `config`. Elle comprend un NameNode, deux DataNodes, un ResourceManager et deux NodeManagers. Pour démarrer :

```
docker compose up -d --scale datanode=2 --scale nodemanager=2
docker compose ps
docker compose exec namenode bash
```

Les interfaces sont `http://localhost:9870` et `http://localhost:8088`. La variante WSL utilise volontairement 19870 et 18088 pour isoler les ports du TP. Ses deux DataNodes et ses deux NodeManagers sont des processus distincts sur la même machine : cela permet une démonstration pédagogique, sans tolérance à une panne physique de l'hôte.

Exercice 0.1 - Versions

```
java -version
hadoop version
```

La branche Hadoop 3.4 accepte Java 8 ou Java 11 à l'exécution ; ce TP vise Java 11. Les sources Java ciblent le bytecode Java 8 avec `--release 8`. Java 11 a été constaté sur la machine ; la présence de Hadoop 3.4.3 n'a pas été validée.

Exercice 0.2 - Les démons

```
jps
docker compose ps
```

NameNode : HDFS, maître. DataNode : HDFS, esclave. ResourceManager : YARN, maître. NodeManager : YARN, esclave. `Jps` est l'outil d'observation, pas un démon du cluster. En Docker chaque conteneur peut avoir son propre espace de processus : `docker compose ps` donne la vue d'ensemble.

Exercice 0.3 - Lire la configuration

```
hdfs getconf -confKey dfs.replication
hdfs getconf -confKey fs.defaultFS
```

Dans l'option Windows du sujet : réplication 1 et `hdfs://localhost:9000`. Dans son option Docker : réplication 2 et `hdfs://namenode:8020`. La réplication est le nombre demandé de copies de chaque bloc. L'URI désigne le système de fichiers par défaut et l'adresse RPC du NameNode. Dans la variante WSL jointe, l'URI est `hdfs://localhost:19000`.

Exercice 0.4 - Le job pi

```
EX=$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.4.3.jar
hadoop jar "$EX" pi 4 1000
hadoop jar "$EX" pi 4 100000
```

Chaque lancement comporte 4 tâches Map et 1 tâche Reduce, plus l'ApplicationMaster : au moins 6 allocations de containers sans nouvelle tentative ni spéculation. Ces allocations ne sont pas nécessairement simultanées. Le nombre de points passe de 4 000 à 400 000. Le calcul utilise des suites de Halton, déterministes ; davantage de points améliore généralement l'estimation, sans garantir une diminution monotone de l'erreur à chaque taille d'échantillon. Aucune valeur de pi n'a été mesurée ici sur YARN.

Exercice 0.5 - Persistance Docker

```
hdfs dfs -mkdir -p /user/etudiant
printf 'test persistance\n' > /tmp/persistance.txt
hdfs dfs -put /tmp/persistance.txt /user/etudiant/
# Depuis le PC : docker compose stop ; docker compose start
hdfs dfs -cat /user/etudiant/persistance.txt
```

`stop/start` conserve les conteneurs et leurs couches inscriptibles : le fichier reste présent. Avec le Compose du sujet, `down/up` supprime puis recrée les conteneurs : les données HDFS sont perdues, car les dossiers internes NameNode/DataNode n'ont pas de volumes persistants. Les montages `jobs` et `donnees` contiennent les fichiers locaux, pas les blocs HDFS. L'archive fournie n'exécute aucun `down` automatique.

TP 1 - HDFS

Exercice 1.1 - Arborescence

```
hdfs dfs -mkdir -p /user/etudiant/tp1/tmp/a/b
hdfs dfs -ls -R /user/etudiant/tp1
```

L'option `-p` crée tous les parents manquants. `-R` affiche récursivement les répertoires et fichiers.

Exercice 1.2 - Espace occupé

```
hdfs dfs -du -s -h /user/etudiant/tp1/livres
```

Taille logique calculée sur les données : **2 843 966 octets**, environ 2,7 Mio. À réplication effective 2 : **5 687 932 octets**, environ 5,4 Mio de charge utile répliquée. À réplication 1 : 2 843 966 octets. La deuxième colonne de du exprime l'espace HDFS répliqué ; l'occupation physique du système hôte comprend aussi des métadonnées et des sommes de contrôle.

Exercice 1.3 - Calcul de blocs

Avec les unités binaires implicites du sujet (1 Gio = 1 024 Mio), des blocs de 128 Mio et r=3 :

Taille du fichier	Blocs	Espace de données répliquées
1 Gio	8	3 Gio
129 Mio	2 : 128 + 1 Mio	387 Mio
1 Kio	1 de 1 Kio	3 Kio

Formules : `nombre de blocs = plafond(T/B)` et `espace = T × r`. Le dernier bloc n'est pas rempli artificiellement.

Exercice 1.4 - Dernier bloc de la Bible

```
hdfs dfs -D dfs.blocksize=1048576 -put /opt/donnees/livres/bible-kjv.txt /user/etudiant/tp1/
hdfs fsck /user/etudiant/tp1/bible-kjv.txt -files -blocks -locations
```

Le fichier contient réellement **4 332 554 octets**. Le calcul donne 5 blocs : quatre de 1 048 576 octets et un dernier de **138 250 octets**. À r=2, cela représente 10 répliques de blocs, si les deux DataNodes sont disponibles. Le nombre de blocs est calculé ici ; `fsck` doit le confirmer sur le cluster.

Exercice 1.5 - Copier, renommer, supprimer

```
hdfs dfs -cp /user/etudiant/tp1/livres/carroll-alice.txt /user/etudiant/tp1/archives/
hdfs dfs -mv /user/etudiant/tp1/archives/carroll-alice.txt /user/etudiant/tp1/archives/alice_2026.txt
hdfs dfs -rm -r /user/etudiant/tp1/tmp
```

La copie conserve l'original ; le renommage porte uniquement sur la copie. La suppression est limitée au répertoire `tmp` créé pour le TP.

Exercice 1.6 - Réplication

```
hdfs dfs -setrep -w 1 /user/etudiant/tp1/livres/austen-emma.txt
hdfs dfs -setrep 3 /user/etudiant/tp1/livres/austen-emma.txt
hdfs fsck /user/etudiant/tp1/livres/austen-emma.txt -files -blocks -locations
hdfs dfs -setrep -w 2 /user/etudiant/tp1/livres/austen-emma.txt
```

Avec deux DataNodes, trois copies distinctes sont impossibles. La réplication demandée devient 3, mais les blocs restent sous-répliqués avec au plus deux copies effectives. Ne pas employer `-w` pour la demande à 3 : l'attente ne pourrait aboutir sans troisième DataNode. Le script restaure ensuite r=2.

Exercice 1.7 - Fusion

```
hdfs dfs -getmerge /user/etudiant/tp1/livres tous.txt
wc -c tous.txt
```

Avant l'ajout de la ligne finale, le fichier fusionné doit contenir **2 843 966 octets**, sans séparateur ajouté par défaut. La fusion locale indépendante est fournie dans les résultats locaux.

Exercice 1.8 - Ajout en fin de fichier

```
printf 'THE END\n' > fin.txt
hdfs dfs -appendToFile fin.txt /user/etudiant/tp1/livres/carroll-alice.txt
hdfs dfs -tail /user/etudiant/tp1/livres/carroll-alice.txt
```

L'ajout utilise 8 octets en UTF-8 avec fin de ligne Unix. Alice passe de 144 395 à 144 403 octets. La commande écrit directement dans HDFS. Les comptages de référence doivent être réalisés avant cet ajout, sinon `the` et `end` gagnent chacun une occurrence.

Exercice 1.9 - Quota

```
hdfs dfs -mkdir -p /user/etudiant/tp1/quota
hdfs dfsadmin -setQuota 3 /user/etudiant/tp1/quota
hdfs dfs -cp /user/etudiant/tp1/livres/carroll-alice.txt /user/etudiant/tp1/quota/
hdfs dfs -cp /user/etudiant/tp1/livres/austen-emma.txt /user/etudiant/tp1/quota/
hdfs dfs -cp /user/etudiant/tp1/livres/shakespeare-hamlet.txt /user/etudiant/tp1/quota/
hdfs dfs -count -q /user/etudiant/tp1/quota
```

Deux fichiers seulement peuvent être copiés : le répertoire quota compte lui-même pour un nom. Le troisième dépasse 3 noms et doit provoquer `NSQuotaExceededException`. C'est un quota de noms, distinct d'un quota d'espace.

Exercice 1.10 - Mode sécurisé

```
hdfs dfsadmin -safemode enter
hdfs dfs -mkdir /user/etudiant/tp1/interdit
hdfs dfs -head /user/etudiant/bonjour.txt
hdfs dfsadmin -safemode leave
```

En mode sécurisé, la création échoue ; la lecture reste possible si les blocs sont accessibles. Ce mode protège les métadonnées contre les modifications. Le script inclut une sortie de secours pour quitter le safe mode même si un contrôle échoue.

TP 2 - YARN

Suivre une application et ses logs

```
yarn node -list -all
yarn node -status <nodeId>
hadoop jar "$EX" pi 16 1000000 > pi.log 2>&1 &
yarn application -list
yarn application -status <applicationId>
yarn applicationattempt -list <applicationId>
yarn container -list <attemptId>
mapred job -list
yarn logs -applicationId <applicationId> > pi_logs.txt
grep -o 'container_[0-9_]*' pi_logs.txt | sort -u | wc -l
```

Les identifiants entre chevrons doivent être remplacés par ceux de l'exécution réelle. Aucun identifiant d'application fictif n'est fourni comme preuve.

Exercice 2.1 - Ressources du cluster

Avec la configuration du TP : 2 NodeManagers × 4 096 Mio = **8 192 Mio**, et 2 × 8 = **16 vcores** déclarés à YARN. À vérifier avec `yarn node -status` ou l'interface Nodes. Ces valeurs sont celles de la configuration, pas nécessairement les ressources physiques disponibles de l'hôte.

Exercice 2.2 - Vagues

Des containers de 1 024 Mio permettent théoriquement 8 allocations simultanées sur le cluster. L'AM en utilise une, donc au plus 7 Map en parallèle pour un seul job. Pour 16 Map, il faut au moins 3 vagues : 7 + 7 + 2. C'est une borne idéale : files d'attente, autres applications, vcores, localisation et début du Reduce peuvent réduire le parallélisme observé.

Exercice 2.3 - États

Cycle interne nominal : `NEW` → `NEW_SAVING` → `SUBMITTED` → `ACCEPTED` → `RUNNING` → `FINISHED`. Les états transitoires ne sont pas tous visibles dans les commandes client. `State` décrit le cycle YARN ; `Final-State` est le verdict de l'application : `SUCCEEDED`, `FAILED` ou `KILLED`, et `UNDEFINED` avant son verdict. Une application peut être `FINISHED` tout en ayant le verdict `FAILED`.

Exercice 2.4 - Arrêt à 30 %

```
yarn application -status <applicationId>
yarn application -kill <applicationId>
yarn application -status <applicationId>
```

Après propagation de l'arrêt, `State = KILLED` et `Final-State = KILLED`. Les intermédiaires Map ne constituent pas un résultat final réutilisable : ils sont locaux et nettoyés. Avec le committer standard, aucun résultat réussi n'est validé et `_SUCCESS` est absent. Le répertoire de sortie, des éléments temporaires ou des fichiers issus de tentatives peuvent subsister selon le moment de l'arrêt : leur absence totale n'est pas une garantie universelle. Il faut contrôler la sortie avant relance et employer un nouveau chemin.

Exercice 2.5 - Containers et logs

Pour `pi 16 1000000`, sans relance ni spéculation : 1 AM + 16 Map + 1 Reduce = **18 containers** sur la totalité du job. Le premier container de la tentative héberge l'ApplicationMaster. Ce total diffère du maximum de 8 containers simultanés. Le résultat effectif du décompte des logs peut être supérieur en cas de tentatives supplémentaires ou incomplet si les logs ne sont pas encore agrégés.

Exercice 2.6 - File inconnue

```
hadoop jar "$EX" pi -D mapreduce.job.queuename=test 4 1000
```

Avec les seules files `default` et `dev`, la soumission à `test` est rejetée : file inconnue (`unknown queue`). L'application ne peut pas calculer normalement. Le diagnostic exact et la visibilité d'une application échouée dépendent de l'étape du rejet.

Exercice 2.7 - Container de 6 000 Mio

```
hadoop jar "$EX" pi -D mapreduce.map.memory.mb=6000 4 1000
```

Chaque NodeManager n'offre que 4 096 Mio. Les ressources de deux nœuds ne s'additionnent pas pour héberger un seul container. Aucune Map de 6 000 Mio ne peut donc s'exécuter. Selon la limite maximale d'allocation effective du scheduler, la demande est rejetée comme invalide ou reste non allouable ; le blocage à 0 % décrit dans le corrigé du sujet n'est pas garanti dans toutes les configurations. Il faut examiner les diagnostics de l'AM et du RM, puis tuer l'application si elle attend indéfiniment.

Exercice 2.8 - Identifiants

Pour une application MapReduce : `application_X_N` correspond à `job_X_N`. Les deux commandes utilisent le même horodatage et le même numéro séquentiel, avec un préfixe différent. YARN peut également gérer des applications autres que MapReduce, qui n'apparaissent donc pas dans la liste `mapred job`.

File dev - Configuration et essai

La configuration fournie définit `root.queues=default,dev`, `default.capacity=70`, `dev.capacity=30` et `dev.maximum-capacity=50`. Les capacités enfants totalisent 100 %. `dev` bénéficie de 30 % et peut monter à 50 % si les ressources sont disponibles. Ce ne sont pas des réservations permanentes de containers.

```
docker compose up -d --force-recreate resourcemanager
yarn queue -status dev
hadoop jar "$EX" pi -D mapreduce.job.queuename=dev 8 100000
```

La graphie `--force-recreate` corrige la coquille à un seul tiret du PDF. Sous installation native, après modification du XML : `yarn rmadmin -refreshQueues`.

WordCount standard fourni par Hadoop

```
hadoop jar "$EX" wordcount /user/etudiant/tp1/livres /user/etudiant/tp2/wc
hdfs dfs -cat /user/etudiant/tp2/wc/part-r-* | sort -k2,2nr | head -3
```

La vérification locale du découpage sur les espaces donne `the 25072`, `of 13789`, `and 13508`. Ce WordCount conserve casse et ponctuation, contrairement au programme nettoyé du TP 3.

TP 3 - MapReduce

Projet Java et principes

Le projet Maven `tp3` dépend de `hadoop-client:3.4.3` en portée `provided`. Les trois classes attendues sont présentes dans les packages du sujet. Le compilateur est explicitement configuré pour Java 8. La compilation s'effectue avec `mvn package` ou le script `compiler.sh` sur une distribution Hadoop disponible. Les dépendances et le binaire Hadoop ne sont pas inclus dans l'archive.

WordCount : le Mapper normalise en minuscules, découpe selon les lettres Unicode et émet `(mot, 1)`. Le Shuffle rassemble les clés identiques. Le Reducer somme. Le Combiner effectue une somme locale, opération associative et commutative. Les options sont `wc.longueur.min`, `wc.mots.vides` et `wc.combiner`.

Méthode amis 1 : chaque extrémité d'une arête émet la même paire triée et sa liste d'amis. Le Reducer fait l'intersection, uniquement si les deux listes sont présentes. Méthode 2 : chaque personne P émet `(groupe de k de ses amis, P)` ; le Reducer rassemble les amis communs. `amis.min` filtre le nombre de personnes reçues. `amis.statut=true` charge le graphe via le cache distribué.

Le code nettoie les espaces, ignore les voisins vides et les boucles sur soi, et déduplique les voisins. Le statut exige la réciprocité de chaque amitié. Le graphe original est symétrique ; avec des données asymétriques, la méthode 2 conserve la sémantique du sujet : une personne qui se déclare amie de tous les membres est comptée. Elle ne transforme pas automatiquement le graphe en réseau symétrique.

Résultats WordCount réellement recalculés

Mot	Occurrences
the	26 940
and	15 283
of	14 119
to	12 672
a	11 141
in	8 382
i	7 531
it	7 048
that	6 525
was	5 630

Total : **511 244 occurrences**, **23 892 mots distincts**. La somme du nombre de clés distinctes par livre vaut **39 051**. Elle correspond au volume combiné de référence, lorsque chaque fichier est combiné en une agrégation complète ; les compteurs Hadoop effectifs peuvent varier avec les spills et les splits.

Avec longueur minimale 3 et la liste de mots vides fournie : **283 390 occurrences ignorées**, donc **227 854 conservées**. Le top 5 est `whale 1228`, `emma 865`, `every 716`, `most 687`, `though 669`. Tous les résultats complets sont joints en TSV, triés par clé.

Exercice 3.1 - Avec et sans Combiner

```
hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.wordcount.WordCountLivres /user/etudiant/tp1/livres
/user/etudiant/tp3/wc
hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.wordcount.WordCountLivres -D wc.combiner=false
/user/etudiant/tp1/livres /user/etudiant/tp3/wc_sans_combiner
```

Les résultats finaux doivent être identiques. Sans Combiner, les 511 244 paires Map sont transmises ; la référence avec Combiner est 39 051 paires, soit **92,36 % de paires en moins**, un rapport de 13,09. Le PDF annonce environ 4,7 Mo contre 0,55 Mo de `Reduce shuffle bytes` : ce sont des valeurs de référence du sujet, non mesurées ici. Le rapport d'octets diffère du rapport des paires à cause du format des enregistrements. La durée dépend du démarrage des tâches, des disques et du réseau ; un gain n'est pas garanti pour un si petit corpus. Comparer les logs et vérifier l'identité des fichiers de sortie avant de conclure.

Exercice 3.2 - Nombre de tâches

Le `TextInputFormat` standard ne crée pas un split qui chevauche deux fichiers. Avec cinq petits fichiers inférieurs à la taille d'un split, il crée cinq splits : **5 Map**. Pour les 18 livres Gutenberg également petits : **18 Map**. Cela suppose le format et les tailles de split du TP ; un autre InputFormat, des fichiers plus grands ou une configuration différente peuvent modifier le nombre de tâches.

Exercice 3.3 - Paires émises

Les degrés du petit graphe sont `(3, 4, 4, 4, 3)`. Une personne de degré d émet `C(d,k)` groupes.

k	Calcul	GROUPES_EMIS attendu	Groupes distincts calculés
2	3 + 6 + 6 + 6 + 3	24	10
3	1 + 4 + 4 + 4 + 1	14	10
4	0 + 1 + 1 + 1 + 0	3	3

Le programme local confirme ces nombres en énumérant les combinaisons. Le compteur Hadoop `Amis/GROUPES_EMIS` doit être relevé après exécution des JAR pour disposer d'une preuve MapReduce.

Petit graphe - Résultats de la méthode 1

```
Amina,Bilal      [Chaima, Driss]
Amina,Chaima     [Bilal, Driss]
Amina,Driss      [Bilal, Chaima]
Bilal,Chaima     [Amina, Driss, Elias]
Bilal,Driss      [Amina, Chaima, Elias]
Bilal,Elias      [Chaima, Driss]
Chaima,Driss     [Amina, Bilal, Elias]
Chaima,Elias     [Bilal, Driss]
Driss,Elias      [Bilal, Chaima]
```

La méthode 1 produit neuf arêtes. Amina et Elias n'apparaissent pas, car ils ne sont pas amis directs.

Petit graphe - Méthode 2, k=2

Elle retrouve les neuf résultats précédents, et ajoute `Amina,Elias → [Bilal, Chaima, Driss]`, avec le statut `PAS tous amis`. Les neuf autres paires ont le statut `amis entre eux`. Le fichier `amis_k2.tsv` contient les dix lignes complètes.

Petit graphe - Méthode 2, k=3

```
Amina,Bilal,Chaima  1 [Driss]          amis entre eux
Amina,Bilal,Driss   1 [Chaima]         amis entre eux
Amina,Bilal,Elias   2 [Chaima, Driss]  PAS tous amis
Amina,Chaima,Driss  1 [Bilal]          amis entre eux
Amina,Chaima,Elias  2 [Bilal, Driss]   PAS tous amis
Amina,Driss,Elias   2 [Bilal, Chaima]  PAS tous amis
Bilal,Chaima,Driss  2 [Amina, Elias]   amis entre eux
Bilal,Chaima,Elias  1 [Driss]          amis entre eux
Bilal,Driss,Elias   1 [Chaima]         amis entre eux
Chaima,Driss,Elias  1 [Bilal]          amis entre eux
```

Petit graphe - Méthode 2, k=4

```
Amina,Bilal,Chaima,Elias 1 [Driss]  PAS tous amis
Amina,Bilal,Driss,Elias  1 [Chaima] PAS tous amis
Amina,Chaima,Driss,Elias 1 [Bilal]  PAS tous amis
```

Les deux autres groupes de quatre n'ont aucun ami commun et ne sont pas émis. La méthode 2 ne matérialise pas les groupes à intersection vide, même en l'absence de filtre supplémentaire.

Exercice 3.4 - Amitié à sens unique

Le fichier `amis_asymetrique.txt` ajoute `Farid:Amina,Bilal`, sans réciprocité. Méthode 1 : `Amina,Bilal → [Chaima, Driss]`, inchangé, et deux arêtes à sens unique (Amina-Farid et Bilal-Farid) sont ignorées ; le compteur attendu est 2. Méthode 2, k=2 : `Amina,Bilal → [Chaima, Driss, Farid]`, car Farid se déclare ami des deux. Le fichier de résultat asymétrique recalculé est joint.

Exercice 3.5 - Plus grand groupe sur 1 000 personnes

```
hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.amis.AmisCommunsK -D amis.k=3 -D amis.min=3 -D
amis.statut=true /user/etudiant/tp3/amis_1000.txt /user/etudiant/tp3/grand_k3
hdfs dfs -cat /user/etudiant/tp3/grand_k3/part-r-* | sort -k2,2nr | head
```

Le calcul indépendant trouve **Driss00723,Meryem00747,Nabil00712**, avec **8 amis communs**, et le statut **PAS tous amis**. Le résultat complet permet de retrouver les huit noms dans la liste associée. Le maximum pour k=3 est unique dans ce jeu de données.

Passage à l'échelle réellement recalculé

k	Groupes émis	Groupes distincts	Avec au moins 3 amis communs
2	239 145	159 525	17 705
3	1 704 421	1 591 428	19 284
4	8 974 079	8 869 719	6 163

Ces valeurs correspondent exactement au sujet. Les durées du script Python sont enregistrées dans `synthese.json` ; elles ne sont pas comparables aux durées MapReduce du PDF. Pour d=22 : `C(22,2)=231`, `C(22,3)=1540`, `C(22,4)=7315`. Le filtre `amis.min` agit après l'émission : il réduit la sortie mais n'évite pas l'explosion du Shuffle. Le fichier de 10 000 personnes est conservé comme bonus fourni ; son traitement n'est pas requis par les exercices et n'a pas été lancé.

Reproduction et fichiers livrés

1. Lire `LISEZMOI.md` et la présente limite d'exécution. 2. Démarrer le cluster Docker ou installer Hadoop 3.4.3 dans la variante WSL. 3. Compiler les trois classes Java, puis placer `tp3.jar` dans `jobs`. 4. Exécuter `scripts/tp_cluster.sh` sur un espace HDFS de TP neuf. Le script n'est pas idempotent : plusieurs sorties MapReduce doivent être nouvelles. 5. Pour les manipulations interactives de suivi/arrêt à 30 %, utiliser deux terminaux et les commandes du TP 2 ; elles ne sont pas automatisées dans le script principal. 6. Vérifier la persistance `stop/start` séparément ; l'essai destructif `down/up` est expliqué mais n'est pas lancé automatiquement. 7. Comparer les résultats Hadoop avec les TSV de `resultats/verification_locale` et recueillir les captures NameNode/YARN sur le cluster opérationnel.

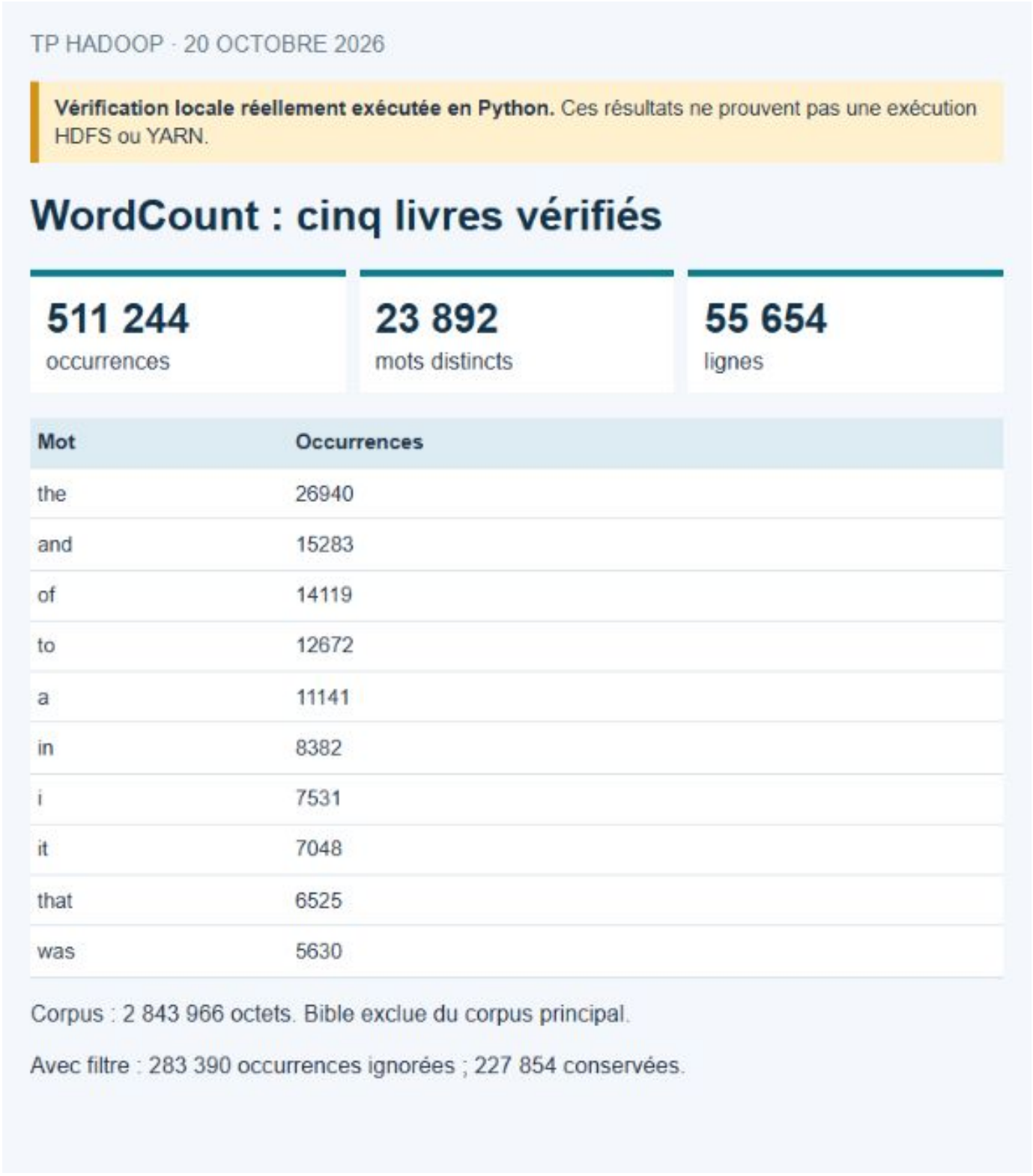
Les fichiers de résultat local sont des données réellement calculées, pas des sorties portant artificiellement `_SUCCESS` ou des identifiants YARN. Les scripts conservent les logs réels quand ils sont exécutés. Le manifeste SHA-256 permet de vérifier l'intégrité des fichiers de l'archive.

Références

Sujet original fourni : H. BADIR AI, Hadoop - Travaux pratiques TP 0 à TP 3, édition 2026, 38 pages. Données : corpus Gutenberg et réseaux synthétiques fournis avec le TP.

Documentation officielle : https://hadoop.apache.org/docs/r3.4.3/ ; guide du Capacity Scheduler : https://hadoop.apache.org/docs/r3.4.2/hadoop-yarn/hadoop-yarn-site/CapacityScheduler.html ; versions Apache Hadoop : https://hadoop.apache.org/release/index.html . Les descriptions de quotas, splits, ressources et limites doivent être lues avec la configuration effective du cluster.

Capture - 01 wordcount



TP HADOOP · 20 OCTOBRE 2026

Vérification locale réellement exécutée en Python. Ces résultats ne prouvent pas une exécution HDFS ou YARN.

WordCount : cinq livres vérifiés

511 244	23 892	55 654
occurrences	mots distincts	lignes

Mot	Occurrences
the	26940
and	15283
of	14119
to	12672
a	11141
in	8382
i	7531
it	7048
that	6525
was	5630

Corpus : 2 843 966 octets. Bible exclue du corpus principal.

Avec filtre : 283 390 occurrences ignorées ; 227 854 conservées.

Capture réelle du navigateur affichant les résultats de vérification locale. Elle ne représente pas une interface Hadoop exécutée.

Capture - 02 amis

TP HADOOP · 20 OCTOBRE 2026

> **Vérification locale réellement exécutée en Python.** Ces résultats ne prouvent pas une exécution HDFS ou YARN.

Amis communs : petit réseau

k = 2 : dix groupes

Groupe	Nombre	Amis communs	Statut
Amina,Bilal	2	[Chaima, Driss]	amis entre eux
Amina,Chaima	2	[Bilal, Driss]	amis entre eux
Amina,Driss	2	[Bilal, Chaima]	amis entre eux
Amina,Elias	3	[Bilal, Chaima, Driss]	PAS tous amis
Bilal,Chaima	3	[Amina, Driss, Elias]	amis entre eux
Bilal,Driss	3	[Amina, Chaima, Elias]	amis entre eux
Bilal,Elias	2	[Chaima, Driss]	amis entre eux
Chaima,Driss	3	[Amina, Bilal, Elias]	amis entre eux
Chaima,Elias	2	[Bilal, Driss]	amis entre eux
Driss,Elias	2	[Bilal, Chaima]	amis entre eux

Compteurs calculés : 24 / 14 / 3 émissions pour k = 2 / 3 / 4

Méthode 1 : neuf arêtes ; méthode 2 : Amina-Elias apparaît avec trois amis communs.

Capture réelle du navigateur affichant les résultats de vérification locale. Elle ne représente pas une interface Hadoop exécutée.

Capture - 03 grand graphe

TP HADOOP · 20 OCTOBRE 2026

> **Vérification locale réellement exécutée en Python.** Ces résultats ne prouvent pas une exécution HDFS ou YARN.

Passage à l'échelle : 1 000 personnes

k	Groupes émis	Distincts	Avec ≥ 3 amis communs
2	239145	159525	17705
3	1704421	1591428	19284
4	8974079	8869719	6163

Exercice 3.5 : maximum pour k = 3

```
Driss00723,Meryem00747,Nabil00712       8       [Anas00744, Elias00724,
Ghita00726, Ismail00716, Kenza00730, Omar00734, Youssef00742, Zineb00715]
PAS tous amis
```

Le groupe possède huit amis communs. Ses membres ne sont pas tous amis entre eux.

Les trois volumes correspondent aux valeurs du sujet. Les résultats complets sont conservés en TSV.

Capture réelle du navigateur affichant les résultats de vérification locale. Elle ne représente pas une interface Hadoop exécutée.

Capture - 04 filtre



TP HADOOP · 20 OCTOBRE 2026

Vérification locale réellement exécutée en Python. Ces résultats ne prouvent pas une exécution HDFS ou YARN.

WordCount : nettoyage et contrôle

Mot conservé	Occurrences
whale	1228
emma	865
every	716
most	687
though	669

283 390
mots ignorés

227 854
mots conservés

39 051
clés distinctes cumulées par livre

WordCount standard : découpage sur les espaces

Mot	Occurrences
the	25072
of	13789
and	13508

Programme de contrôle : scripts/verifier_local.py. Données : donnees/livres, cinq livres explicitement sélectionnés.

Capture réelle du navigateur affichant les résultats de vérification locale. Elle ne représente pas une interface Hadoop exécutée.